IT STUDY HUB

COMPLETE C++ PROGRAMMING SERIES

MODULE 6: EXCEPTION HANDLING, FILE HANDLING & STL BASICS

Authors: Mithila Rani Saha & Dithsa Dutta

Website: www.itstudyhub.dpdns.org

Module Overview

1. About This Module

Welcome to **Module 6: Exception Handling, File Handling & STL Basics** in the IT Study Hub Complete C++ Programming Series. Building upon the core object-oriented principles mastered in Module 5, this module arms you with robust error

management techniques, permanent persistent storage mechanisms via file streams, and rapid application development components using the Standard Template Library (STL). Authored by Mithila Rani Saha and Dithsa Dutta, this handbook ensures your programs are bulletproof, persistent, and highly efficient.

2. Learning Outcomes

By completing this module, you will be able to:

- Handle runtime errors gracefully using `try`, `catch`, `throw`, multiple catch blocks, and custom exception classes.
- Perform file input/output operations using `ifstream`, `ofstream`, and `fstream` for text and binary file persistence.
- Accept and process command-line arguments (`argc`, `argv`) in C++ applications.
- Understand the core architecture of the Standard Template Library (STL) including containers, iterators, and algorithms.
- Utilize fundamental STL data structures like `std::vector` and `std::pair` in practical engineering tasks.

3. Module Coverage

This module comprehensively covers:

- **Exception Handling:** `try`, `catch`, `throw`, exception hierarchies, multiple catches, and custom exceptions.
- **File I/O Streams:** `ifstream`, `ofstream`, `fstream`, opening/closing, text vs binary files, reading, writing, and appending data.
- **Command-Line Arguments:** Passing parameters to `main(int argc, char* argv[])`.
- **STL Architecture:** Containers, Iterators, and Algorithms.
- **Essential STL Data Structures:** `std::vector` and `std::pair`.
- **Hands-On Project:** Building a robust File-Based Student Record Management System.

- **Rigorous Assessments:** 15+ practice questions, 20+ viva questions with answers, 15+ technical interviews, and coding challenges.

IT STUDY HUB

4. Main Notes: Theoretical Foundations

Professional software must anticipate runtime failures, persist data to disk, and utilize pre-tested standard libraries. Below are the exhaustive foundational notes for Module 6.

Concept: Exception Handling: try, catch, throw, and Multiple Catch Blocks

Simple Explanation: Exception handling provides a structured mechanism to detect and respond to runtime errors (exceptions) rather than letting programs crash abruptly. The `try` block encloses risky code, `throw` signals an error condition, and `catch` blocks handle the thrown exception based on data type.

> ***Real-Life Analogy:*** *Think of exception handling like an airplane's emergency oxygen mask system. The normal flight path runs inside the `try` block. If cabin pressure drops (an error occurs), an oxygen mask is `throw`n down. Passengers put on the mask (`catch`), stabilizing the plane instead of crashing.*

```
Class Diagram / Architecture:
try { riskyCode(); throw error; } → catch (exception e) {
handle(); }
```

```
Syntax:
try {
if (err) throw exceptionObj;
}
catch (type1 e) {
// handle type1
}
catch (...) {
```

```
// catch-all handler
}
```

```
#include
using namespace std;

int main() {
    double num1, num2;
    cout << "Enter numerator and denominator: ";
    cin >> num1 >> num2;

    try {
        if (num2 == 0) {
            throw string("Division by zero error!");
        }
        cout << "Result: " << num1 / num2 << endl;
    }
    catch (string &err) {
        cout << "Caught Exception: " << err << endl;
    }
    catch (...) {
        cout << "Caught unknown exception." << endl;
    }

    cout << "Program continues normally after try-catch." << endl;
    return 0;
}
```

```
Output:
Enter numerator and denominator: 10 0
Caught Exception: Division by zero error!
Program continues normally after try-catch.
```

Line-by-Line Breakdown:

- **try { ... }**: Monitors code for potential exceptions.
- **throw string(...)**: Throws an exception object when denominator is zero.
- **catch (string &err)**: Catches the string exception and executes recovery logic.

Real-World Application: Network socket connection timeouts and database query failures handled gracefully in enterprise backend software.

> **Common Mistakes:** Catching exceptions by value instead of by reference (`catch (exception e)` vs `catch (const exception &e)`), which causes object slicing.

> **Best Practices:** Always catch standard exceptions by `const reference` to avoid unnecessary object copying and slicing.

> **Actionable Practice:** Write a try-catch block that validates user age and throws an integer error if age is negative.

Concept: Custom Exceptions

Simple Explanation: C++ allows creating custom exception classes by inheriting from the standard library's `` base class (or `std::runtime_error`, `std::logic_error`), overriding the `what()` method to provide descriptive error messages.

> ***Real-Life Analogy:*** *A custom exception is like a specialized alarm code tailored for a specific factory machine failure (e.g., 'Overheating Alarm Code 402') rather than a generic building fire alarm.*

```
Class Diagram / Architecture:
std::exception → CustomException : public runtime_error → what() overridden
```

```
Syntax:
class MyException : public exception {
public:
```

```
const char* what() const noexcept override {
return "Custom error occurred";
}
};
```

```
#include
#include
using namespace std;

class InsufficientFundsException : public exception {
    public:
        const char* what() const noexcept override {
            return "Error: Withdrawal amount exceeds available
balance!";
        }
};

int main() {
    double balance = 500.0;
    double withdrawal = 800.0;

    try {
        if (withdrawal > balance) {
            throw InsufficientFundsException();
        }
        balance -= withdrawal;
    }
    catch (const InsufficientFundsException &e) {
        cout << e.what() << endl;
    }
    return 0;
}
```

```
Output:
Error: Withdrawal amount exceeds available balance!
```

Line-by-Line Breakdown:

- **class InsufficientFundsException : public exception**: Inherits standard exception base.
- **what() const noexcept override**: Overrides error description message safely.

• **throw InsufficientFundsException()**: Throws custom exception instance.

Real-World Application: Banking transaction engines throwing specialized domain-specific exceptions like `AccountLockedException` or `DailyLimitExceededException`.

> **Common Mistakes:** Forgetting the `noexcept` specifier on `what()` overrides, violating standard exception interface expectations.

> **Best Practices:** Derive custom exceptions from standard library exception classes (`std::runtime_error`) rather than raw primitive types.

> **Actionable Practice:** Create a custom `InvalidPasswordException` class and throw it when password length is below 8 characters.

Concept: File Handling: ifstream, ofstream, fstream, and File Streams

Simple Explanation: File handling enables persistent data storage on disk disks. C++ provides file stream classes in the `` header: `ofstream` for writing files, `ifstream` for reading files, and `fstream` for bidirectional read/write operations.

> ***Real-Life Analogy:*** *File streams are like physical pipelines connected between your program (RAM) and a storage vault on your hard drive. `ofstream` pumps data out to the vault, `ifstream` pumps data in from the vault.*

```
Class Diagram / Architecture:
Program (RAM) → ofstream → Hard Drive File | Hard Drive File → ifstream → Program (RAM)
```

```
Syntax:
#include
ofstream outFile("file.txt");
ifstream inFile("file.txt");
fstream ioFile("file.txt", ios::in | ios::out);
```

```
#include
#include
using namespace std;

int main() {
    // Writing to file
    ofstream outFile("it_study_hub.txt");
    if (outFile.is_open()) {
        outFile << "Welcome to IT Study Hub File Handling!\n";
        outFile << "C++ Programming Series Module 6.\n";
        outFile.close();
        cout << "Data written successfully.\n";
    } else {
        cout << "Error opening file for writing.\n";
    }
    return 0;
}
```

```
Output:
Data written successfully.
```

Line-by-Line Breakdown:

- **#include <fstream>**: Required header for file stream operations.
- **ofstream outFile("...");**: Creates and opens a file for output writing.
- **outFile.is_open()**: Verifies file opened successfully before writing.
- **outFile.close()**: Explicitly flushes and closes file stream.

Real-World Application: Saving game progress states, application logs, and database transaction records to disk.

Common Mistakes: Failing to check whether `is_open()` returns true before reading or writing to a file stream.

Best Practices: Always close file streams explicitly or rely on RAII destructors to close files when streams go out of scope.

Actionable Practice: Write a program that creates a text file and writes your name and course title into it.

Concept: Reading, Writing, Appending, and Binary Files

Simple Explanation: Files can be opened in different modes (`ios::out`, `ios::in`, `ios::app` for appending, `ios::binary`). Text files store human-readable characters, while binary files store raw memory byte representations directly.

> ***Real-Life Analogy:*** *Appending data is like adding a new log entry to the bottom of a continuous ledger book rather than erasing and rewriting the whole page. Binary files are like packing items tightly into sealed crates without paper wrappers.*

```
Class Diagram / Architecture:
ios::out (Overwrite) | ios::app (Append at end) | ios::binary (Raw byte stream)
```

```
Syntax:
ofstream file("log.txt", ios::app); // Append mode
fstream binFile("data.bin", ios::out | ios::binary);
```

```
#include
#include
using namespace std;

int main() {
    // Appending data
    ofstream appendFile("it_study_hub.txt", ios::app);
    appendFile << "Appending new log entry.\n";
    appendFile.close();

    // Reading file back
    ifstream inFile("it_study_hub.txt");
    string line;
    cout << "--- File Contents ---:\n";
    while (getline(inFile, line)) {
        cout << line << endl;
    }
    inFile.close();
    return 0;
}
```

> **Output:**
> ```
> --- File Contents ---:
> Welcome to IT Study Hub File Handling!
> C++ Programming Series Module 6.
> Appending new log entry.
> ```

Line-by-Line Breakdown:

- **ios::app**: Opens file in append mode, preserving existing content.
- **getline(inFile, line)**: Reads file line-by-line until EOF (End Of File).

Real-World Application: Application logging systems appending timestamped error entries to log files.

> **Common Mistakes:** Reading binary files using text stream operators (`>>` or `getline`), which corrupts raw byte parsing.

> **Best Practices:** Use `read()` and `write()` methods with `.bat` structs when handling binary file serialization.

> **Actionable Practice:** Write a program that opens an existing file in append mode and adds three new lines of text.

Concept: Command-Line Arguments (argc, argv)

Simple Explanation: C++ allows passing arguments from the operating system shell directly into the program via `main(int argc, char* argv[])`. `argc` represents argument count, and `argv` is an array of C-style character strings holding the arguments.

> ***Real-Life Analogy:*** *Command-line arguments are like giving launch coordinates to a rocket before pressing the start button: `program.exe input.txt output.txt`.*

> **Class Diagram / Architecture:**
> **Terminal: `./app config.json 5` → argc = 3 → argv[0] = "./app", argv[1] = "config.json", argv[2] = "5"**

> **Syntax:**
> **int main(int argc, char *argv[]) {**
> **// argv[0] is program path**
> **// argv[1] is first argument**
> **}**

```
#include
using namespace std;

int main(int argc, char *argv[]) {
```

```
    cout << "Total arguments (argc): " << argc << endl;
    for(int i = 0; i < argc; i++) {
        cout << "argv[" << i << "] = " << argv[i] << endl;
    }
    return 0;
}
```

Output:
```
Total arguments (argc): 1
argv[0] = ./a.out (or executable path)
```

Line-by-Line Breakdown:

- **int argc**: Counts how many arguments were passed from CLI.
- **char *argv[]**: Array of pointers holding argument strings.

Real-World Application: CLI utilities like compilers (`g++ main.cpp`) and build tools (`make`, `git clone`) accepting command-line flags.

Common Mistakes: Accessing `argv[1]` without checking if `argc > 1`, causing segmentation faults when no arguments are provided.

Best Practices: Always validate `argc` bounds before accessing elements in `argv` arrays.

Actionable Practice: Write a CLI program that takes two numbers as command-line arguments, converts them using `stoi`, and prints their sum.

Concept: Introduction to STL: Containers, Iterators, and Algorithms

Simple Explanation: The Standard Template Library (STL) is a powerful collection of generic classes and functions built around three core pillars: Containers (data structures like vector, list, map), Iterators (pointer-like objects traversing containers), and Algorithms (sorting, searching, manipulation functions).

> ***Real-Life Analogy:*** *Think of STL as a fully equipped tool workshop: Containers are storage bins of various shapes (shelves, drawers), Iterators are robotic gripper arms that reach into bins to grab items, and Algorithms are specialized power tools (saws, sorters) that process the items inside bins.*

> **Class Diagram / Architecture:**
> **Containers (Storage) → Iterators (Traversal) → Algorithms (Processing)**

> **Syntax:**
> **#include**
> **#include**
> **vector v;**
> **sort(v.begin(), v.end());**

```
#include
#include
#include
using namespace std;

int main() {
    vector numbers = {45, 12, 89, 33, 7};

    // Sorting using STL algorithm
    sort(numbers.begin(), numbers.end());

    cout << "Sorted numbers: ";
```

```
    for(auto it = numbers.begin(); it != numbers.end(); ++it) {
        cout << *it << " ";
    }
    cout << endl;
    return 0;
}
```

Output:
```
Sorted numbers: 7 12 33 45 89
```

Line-by-Line Breakdown:

- **vector<int>**: STL dynamic container storing integers.
- **sort(begin, end)**: STL sorting algorithm operating via iterators.
- **vector::iterator**: Iterates through container elements.

Real-World Application: Enterprise data processing pipelines requiring high-speed sorting, searching, and memory-managed collections.

Common Mistakes: Invalidating iterators by modifying container structures (inserting/deleting elements) while traversing them with active iterators.

Best Practices: Use range-based for loops (`for (int x : vec)`) or standard algorithms instead of manual index loops when possible.

Actionable Practice: Create a vector of 5 floats, sort them in descending order, and print them.

Concept: Basic Vector and Pair

Simple Explanation: `std::vector` is a sequence container representing a dynamic array that automatically resizes itself when elements are added or removed. `std::pair` is a utility container from `` that couples two heterogeneous values together into a single unit.

> ***Real-Life Analogy:*** *A `std::vector` is like an expandable accordion folder that adds more pockets as documents accumulate. A `std::pair` is like a double-sided luggage tag holding a name on one side and a phone number on the other.*

Class Diagram / Architecture:
std::vector → Dynamic Contiguous Buffer | std::pair → [first | second]

Syntax:
#include
#include
vector vec;
pair p = {"Mithila", 95};

```
#include
#include
#include
using namespace std;

int main() {
    vector> studentGrades;

    studentGrades.push_back(make_pair("Mithila", 92));
    studentGrades.push_back({"Dithsa", 95}); // Initializer list

    cout << "--- Student Grade Pairs ---\n";
    for(const auto &p : studentGrades) {
        cout << p.first << " scored " << p.second << endl;
    }
```

```
    return 0;
}
```

Output:

```
--- Student Grade Pairs ---
Mithila scored 92
Dithsa scored 95
```

Line-by-Line Breakdown:

- **vector<pair<string, int>>**: Vector container holding key-value pairs.
- **push_back(...)**: Adds element dynamically to the vector tail.
- **p.first and p.second**: Accesses pair components.

Real-World Application: Storing coordinate points (`pair`) or associative database query results in memory.

Common Mistakes: Accessing vector elements via `vec[i]` without checking bounds when index might exceed size (use `vec.at(i)` for checked bounds).

Best Practices: Use `emplace_back()` instead of `push_back()` when constructing objects directly inside vectors for optimal performance.

Actionable Practice: Create a vector of pairs storing item names and prices, then calculate total sum.

Visual Workflows & Architecture

Architectural diagrams mapping exception-handling control flow, file stream input/output pipelines, command-line argument mapping, and STL architecture.

Diagram 1: Exception-Handling Flow (`try` → `throw` → `catch`)

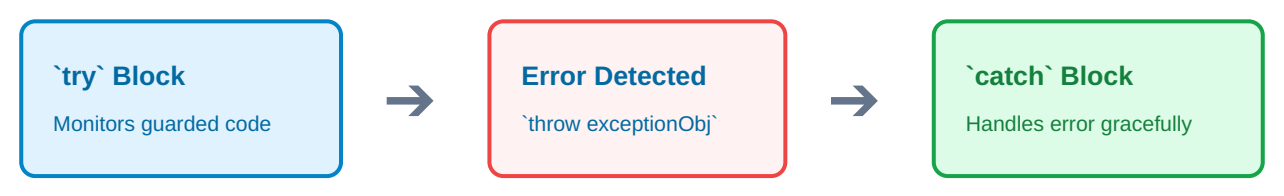


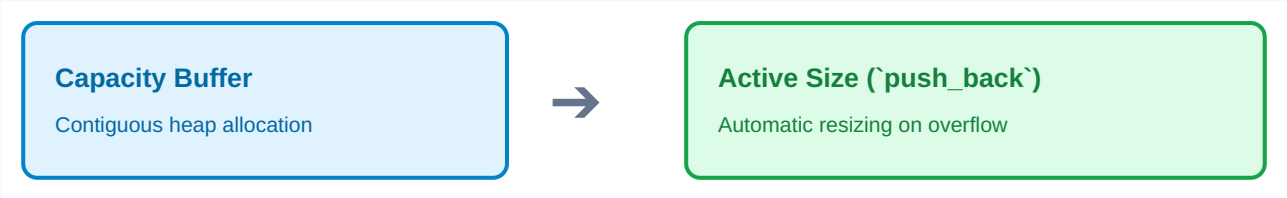
Diagram 2: File I/O Stream Architecture



Diagram 3: STL Architecture (Container → Iterator → Algorithm)



Diagram 4: Vector Dynamic Memory Layout



IT STUDY HUB

Comparative Analysis

Professional C++ software engineering requires mastering the distinctions between error handling mechanisms, file streams, and STL components.

Table 1: Error vs Exception

Aspect	Conventional Error (e.g., Error Codes)	C++ Exception
Propagation	Must be checked manually after every function call.	Propagates automatically up the call stack until caught.
Separation	Mixes error checking directly with business logic.	Cleanly separates business logic (`try`) from error handling (`catch`).

Table 2: ifstream vs ofstream vs fstream

Stream Class	Primary Direction	Default Open Mode
`ifstream`	Input (Reading from file)	`ios::in`
`ofstream`	Output (Writing to file)	`ios::out` (truncates existing file)
`fstream`	Bidirectional (Read and Write)	`ios::in | ios::out`

Table 3: Text vs Binary Files

Feature	Text Files	Binary Files
Readability	Human-readable characters.	Machine-readable raw memory bytes.
Translation	Undergoes newline translations (`\n` to CRLF).	No translation; exact byte-for-byte fidelity.
Performance	Slower due to parsing formatting overhead.	Faster raw memory block transfer.

Table 4: vector vs array

Feature	Built-in Array (`int arr[N]`)	`std::vector`
Size	Fixed size determined at compile time.	Dynamic size that grows/shrinks at runtime.
Safety & Bounds	No bounds checking.	Checked access via `.at()` method.
Memory Management	Stack-allocated (usually).	Heap-allocated internal buffer managed automatically.

Table 5: vector vs pair

Aspect	`std::vector`	`std::pair`
Purpose	Sequence container holding multiple elements.	Utility container coupling exactly two values.
Access	Indexed via `[i]` or `.at(i)`.	Accessed via `.first` and `.second`.

HANDS-ON PROJECT: FILE-BASED STUDENT RECORD MANAGEMENT SYSTEM

In this comprehensive hands-on project, we combine exception handling, file streams (`ifstream`, `ofstream`), STL vectors, pairs, iterators, algorithms, and a menu-driven interface into a professional student record management utility.

Project Specification

Problem Statement: Build a complete file-based Student Record Management System that supports adding records, writing/reading from disk files, searching by roll number, updating scores, displaying all records, and handling invalid user input robustly using try-catch exception blocks and STL vectors.

```
#include <iostream>
#include <fstream>
#include <vector>
#include <utility>
#include <algorithm>
#include <exception>
using namespace std;

class RecordNotFoundException : public exception {
    public:
        const char* what() const noexcept override {
            return "Error: Student roll number not found in
records!";
        }
};

class StudentManager {
    private:
        // Vector storing pairs of (RollNumber, Name) and a parallel
vector for Marks
        // Or vector of pairs: pair> -> Roll, (Name, Marks)
        vector>> records;
```

```
        const string filename = "student_records.txt";

    public:
        void loadFromFile() {
            records.clear();
            ifstream inFile(filename);
            if (!inFile.is_open()) return; // File might not exist
yet

            int roll;
            string name;
            double marks;
            while (inFile >> roll >> name >> marks) {
                records.push_back({roll, {name, marks}});
            }
            inFile.close();
        }

        void saveToFile() {
            ofstream outFile(filename);
            for (const auto &rec : records) {
                outFile << rec.first << " " << rec.second.first << "
" << rec.second.second << "
";
            }
            outFile.close();
        }

        void addStudent(int roll, string name, double marks) {
            if (marks < 0 || marks > 100) {
                throw invalid_argument("Error: Marks must be between
0 and 100.");
            }
            records.push_back({roll, {name, marks}});
            saveToFile();
            cout << "Student record added and saved successfully.
\n";
        }

        void displayAll() {
            loadFromFile();
            if (records.empty()) {
                cout << "No student records found.\n";
                return;
            }
            cout << "=======================================\n";
```

```
            cout << "          STUDENT RECORD LIST           \n";
            cout << "========================================\n";
            for (const auto &rec : records) {
                cout << "Roll: " << rec.first << " | Name: " <<
rec.second.first << " | Marks: " << rec.second.second << endl;
            }
        }

        void searchStudent(int roll) {
            loadFromFile();
            auto it = find_if(records.begin(), records.end(), [roll]
(const pair> &p) {
                return p.first == roll;
            });

            if (it != records.end()) {
                cout << "Found! Roll: " << it->first << " | Name: "
<< it->second.first << " | Marks: " << it->second.second << endl;
            } else {
                throw RecordNotFoundException();
            }
        }
};

int main() {
    StudentManager manager;
    int choice;

    do {
        cout << "\n========================================" <<
endl;
        cout << "    IT STUDY HUB RECORD MANAGER         " << endl;
        cout << "========================================" << endl;
        cout << "1. Add Student Record\n2. Display All Records\n3.
Search by Roll Number\n4. Exit\n";
        cout << "Enter choice (1-4): ";

        if (!(cin >> choice)) {
            cout << "Invalid input stream state. Exiting.\n";
            break;
        }

        try {
            switch(choice) {
                case 1: {
                    int roll;
```

```
                    string name;
                    double marks;
                    cout << "Enter Roll Number: ";
                    cin >> roll;
                    cout << "Enter Name: ";
                    cin >> name;
                    cout << "Enter Marks (0-100): ";
                    cin >> marks;
                    manager.addStudent(roll, name, marks);
                    break;
                }
                case 2:
                    manager.displayAll();
                    break;
                case 3: {
                    int roll;
                    cout << "Enter Roll Number to search: ";
                    cin >> roll;
                    manager.searchStudent(roll);
                    break;
                }
                case 4:
                    cout << "Exiting application. Goodbye!\n";
                    break;
                default:
                    cout << "Invalid menu choice. Select between 1
and 4.\n";
            }
        }
        catch (const exception &e) {
            cout << "Exception Caught: " << e.what() << endl;
        }
    } while(choice != 4);

    return 0;
}
```

Expected Output:

```
========================================
    IT STUDY HUB RECORD MANAGER
========================================
1. Add Student Record
2. Display All Records
```

```
3. Search by Roll Number
4. Exit
Enter choice (1-4): 1
Enter Roll Number: 101
Enter Name: Mithila
Enter Marks (0-100): 105
Exception Caught: Error: Marks must be between 0 and 100.
```

Step-by-Step Explanation:

- `StudentManager` encapsulates records using an STL vector of nested pairs: `vector<pair<int, pair<string, double>>>`.
- `loadFromFile()` and `saveToFile()` handle persistent file I/O using `ifstream` and `ofstream`.
- Custom exception `RecordNotFoundException` and standard `invalid_argument` handle errors robustly within `try-catch` blocks.
- STL algorithms (`find_if`) search container elements efficiently via iterators.

Practice Challenges:

- Add an option 5 to update an existing student's marks by roll number.
- Add an option 6 to delete a student record from the vector and rewrite the file.

Module Assessments

Test your mastery of exception handling, file handling, and STL basics with these rigorously curated assessment questions.

15+ Practice Questions

1. What is the purpose of `try`, `catch`, and `throw` in C++ exception handling?
2. Write a program that handles division by zero using standard try-catch blocks.
3. How do you create a custom exception class in C++?
4. What are the differences between `ifstream`, `ofstream`, and `fstream`?
5. Why is it important to check `is_open()` before reading or writing to a file?
6. Explain how file open modes like `ios::app` and `ios::binary` alter stream behavior.
7. What are command-line arguments (`argc`, `argv`), and how do you access them?
8. What are the three core pillars of the Standard Template Library (STL)?
9. How do iterators act as a bridge between containers and algorithms?
10. What is a `std::vector`, and how does it manage dynamic memory automatically?
11. What is a `std::pair`, and when should you use it?
12. Write a program using `std::vector` to store 5 integers and sort them using `std::sort`.
13. What happens if an exception is thrown inside a `try` block but no matching `catch` block exists?
14. Explain object slicing when catching exceptions by value instead of reference.
15. Write a program that reads a text file line-by-line and counts total words.

20+ Viva Questions (With Answers)

Q1: What is exception handling?

> A mechanism to detect and handle runtime errors gracefully without crashing the program.

Q2: What does the `throw` keyword do?

> Signals that an error or exceptional condition has occurred.

Q3: What is the purpose of a catch-all handler (`catch (...)`)?

> To catch any type of exception that is not caught by preceding catch blocks.

Q4: How do you create a custom exception in C++?

> By inheriting from `std::exception` (or standard runtime/logic error classes) and overriding `what()`.

Q5: What header is required for file stream operations?

> `#include <fstream>`.

Q6: What is `ifstream` used for?

> Input file stream—reading data from files.

Q7: What is `ofstream` used for?

> Output file stream—writing data to files.

Q8: What does `ios::app` mode do?

> Opens a file in append mode, preserving existing file contents and writing new data at the end.

Q9: What is `argc` in command-line arguments?

> Argument count—the total number of arguments passed via the command line.

Q10: What is `argv`?

> Argument vector—an array of C-style character strings holding the actual command-line arguments.

Q11: What is the Standard Template Library (STL)?

> A library of container classes, iterators, and algorithms providing standardized data structures.

Q12: What are the three main components of STL?

> Containers, Iterators, and Algorithms.

Q13: What is a container?

> An object that stores a collection of other objects (data structures like vector, list, map).

Q14: What is an iterator?

> An object with pointer-like behavior used to traverse elements inside a container.

Q15: What is `std::vector`?

> A sequence container representing a dynamic array that resizes automatically.

Q16: What is `std::pair`?

> A utility object that couples two heterogeneous values together into a single unit (`first` and `second`).

Q17: How do you add an element to the end of a vector?

> Using the `push_back()` or `emplace_back()` method.

Q18: What is the difference between text and binary files?

> Text files store human-readable characters with newline translations; binary files store raw memory bytes without translation.

Q19: What happens if a file stream destructor executes without `.close()` being called explicitly?

> The stream destructor automatically closes the file and flushes buffers (RAII principle).

Q20: Why should exceptions be caught by reference (`const &`)?

> To prevent object slicing and avoid unnecessary copy overhead.

15+ Technical Interview Questions

1. **How does stack unwinding work during exception propagation?** Explain how local stack objects are destroyed in reverse order of creation when an exception is thrown until a matching catch block is found.

2. **What is RAII (Resource Acquisition Is Initialization), and how does it apply to file streams and memory?** Discuss automatic resource management tied to object scopes.
3. **What is object slicing in exception handling, and how do you prevent it?** Explain why catching polymorphic exceptions by value slices derived members, and how catching by reference (`const &`) preserves polymorphic types.
4. **Why should destructors never throw exceptions? (Preview)** Discuss stack unwinding hazards where throwing during stack unwinding calls `std::terminate()`.
5. **How does `std::vector` manage its internal memory capacity versus size?** Explain amortized constant time complexity ($O(1)$) of `push_back` via capacity doubling.
6. **What is the difference between `std::vector::at()` and `operator[]`?** Contrast checked bounds exception throwing vs unchecked fast access.
7. **How do STL algorithms interact with raw arrays vs vectors?** Explain how raw array pointers act as valid iterators for STL algorithms.
8. **What are input/output stream state flags (`failbit`, `eofbit`, `badbit`), and how do you check them?** Discuss error handling in file streams.
9. **How do you serialize and deserialize a custom class to a binary file?** Explain writing raw object memory bytes (`reinterpret_cast`) vs structured serialization.
10. **What is exception safety (basic, strong, and no-throw guarantees)?** Detail resource management guarantees across function boundaries.
11. **Why is `std::endl` discouraged in high-performance logging loops?** Explain that `endl` forces both a newline insertion AND an explicit stream buffer flush (`flush()`).
12. **How do custom exceptions inherit from `std::exception`, and why is overriding `what()` marked `noexcept`?** Discuss standard exception contract compliance.
13. **What is iterator invalidation in STL containers like `vector`?** Explain how reallocations during vector growth invalidate existing iterators and pointers.
14. **How do command-line arguments handle integer inputs securely?** Discuss parsing strings using `stoi` or `stringstream` within try-catch blocks.
15. **What is the difference between `std::pair` and `std::tuple`? (Preview)** Contrast fixed two-element coupling vs multi-element heterogeneous tuples.

Module 6 Wrap-Up

Key Terms Glossary

- **Exception Handling:** Structured runtime error management using `try`, `catch`, and `throw`.
- **File Stream:** Pipeline classes (`ifstream`, `ofstream`, `fstream`) managing disk I/O.
- **Command-Line Arguments:** Parameters (`argc`, `argv`) passed into `main()`.
- **STL:** Standard Template Library containing containers, iterators, and algorithms.
- **Vector:** Sequence container representing a dynamic array.
- **Pair:** Utility container coupling two heterogeneous values.
- **RAII:** Resource Acquisition Is Initialization memory/resource management.

Quick revision Summary

Module 6 establishes robust software engineering practices. Exception handling (`try`, `catch`, `throw`, custom exceptions) prevents application crashes. File handling (`ifstream`, `ofstream`, `fstream`) provides persistent disk storage for text and binary data. Command-line arguments pass shell parameters into programs. The Standard Template Library (STL) provides reusable containers (`std::vector`), iterators, and algorithms (`std::sort`) for high-performance development.

STL and File-Handling Coding Challenges

Complete these challenges before moving to Module 7:

- **Challenge 1: Binary Log Serializer.T** Write a program that serializes custom struct records into a binary file using `ios::binary`, then reads them back into a `std::vector`.
- **Challenge 2: STL Frequency Counter.T** Write a program that reads a text file, extracts words into a `std::vector`, and uses STL algorithms to sort and count word frequencies.
- **Challenge 3: Custom Exception Validator.T** Create a custom `PasswordStrengthException` class and write a registration function that validates password complexity rules, throwing exceptions on failure.

CONGRATULATIONS ON COMPLETING MODULE 6!

WHAT COMES NEXT:

MODULE 7 — STL & GENERIC PROGRAMMING